National University of Computer and Emerging Sciences

Assignment No. 3

Generative AI

AI4009

Spring 2026

Submission Instructions:

- Submit the complete .rar file, which should be named AI_ASS03_XXF_YYYY, where XX represents the batch and YYYY represents the roll number.
- Submit a Word file containing the GitHub repository link, as well as the Medium and LinkedIn post links
- This is a group assignment (max 2 members allowed).
- Any submission not following the submission instructions will not be evaluated.
- Explore the Datasets completely before starting to code.
- You can explore more datasets if needed and must explain your reasoning of using them but there is a constraint attached datasets must need to be used.
- **Submit your own work. Accept your current limitations. Grow from your mistakes.**
- **Plagiarism is strictly prohibited and lead to Straight Zero (^_^).**

Question 1: Tackling Mode Collapse in Generative Adversarial Networks (GANs)

Objective:

The objective of this question is to design and implement a Generative Adversarial Network (GAN) system and address the problem of mode collapse by improving training stability using advanced techniques. The implementation must include:

- A baseline Deep Convolutional GAN (DCGAN)
- An improved Wasserstein GAN with Gradient Penalty (WGAN-GP)

The system should demonstrate how advanced loss functions improve training stability and diversity of generated images.

1. Environment Setup

- **Platform:** https://www.kaggle.com/
- **Accelerator:** GPU T4 x2 (Dual GPU)
- **Dataset:**
  - Pokemon Sprites: https://www.kaggle.com/datasets/jackemartin/pokemon-sprites
  - Anime Faces (64×64): https://www.kaggle.com/datasets/soumikrakshit/anime-faces

2. Model Architecture

- **Baseline Model: DCGAN**

  Configuration:
  - Input Noise Vector (z): 100-dimensional
  - Image Size: 64 × 64

- **Generator:**
  - Transposed Convolution Layers
  - Batch Normalization
  - ReLU Activation
  - Output Activation: Tanh
- **Discriminator:**
  - Convolutional Layers
  - LeakyReLU Activation
  - Output Activation: Sigmoid

Model Requirements:

- Generate realistic images from noise
- Learn data distribution of training dataset
- Provide baseline performance for comparison

➢ Advanced Model: WGAN-GP

Configuration:

- Replace Discriminator with Critic (no sigmoid)
- Loss Function: Wasserstein Loss
- Gradient Penalty ($\lambda = 10$)
- Critic updates per Generator update: 5

Model Requirements:

- Eliminate mode collapse
- Improve diversity of generated samples
- Ensure stable training dynamics

3. Implementation Tasks

Part 1: Data Preparation

1. Load dataset (Pokemon / Anime Faces)
2. Resize all images to **64 × 64**
3. Normalize images to range **[-1, 1]**
4. Create PyTorch DataLoader

Part 2: Forward Pass

➢ DCGAN

1. Sample random noise vector (z)
2. Generate fake image using Generator
3. Pass real and fake images to Discriminator
4. Compute Binary Cross Entropy loss

- **WGAN-GP**
    1. Generate fake images
    2. Compute critic scores for real and fake images
    3. Calculate Wasserstein loss
    4. Apply Gradient Penalty

Part 3: Training Setup

Loss Function

- **DCGAN**
    - Binary Cross Entropy Loss
- **WGAN-GP:**
    - Wasserstein Loss
    - Gradient Penalty

Optimizer

- Adam
- Learning Rate: 0.0002
- Betas: (0.5, 0.999)

Training Strategy:

- Train DCGAN first
- Then train WGAN-GP
- Compare performance

Required Training Techniques (to fit Kaggle T4×2)

- Mixed Precision (torch.cuda.amp)
- Batch Size: 64 (adjust based on GPU memory)
- Save checkpoints every 5–10 epochs
- Use smaller dataset subset if needed
- Monitor GPU memory usage

4. Visualization Module

Create a utility that shows:

- Generated images from DCGAN
- Generated images from WGAN-GP
- Comparison between both models

Display at least 5-10 generated samples per model.

Deliverables

1. **Jupyter Notebook:**

- Complete PyTorch implementation of DCGAN and WGAN-GP.

2. **Training Logs:**
   - Plot Generator Loss vs Epochs.
   - Plot Discriminator/Critic Loss vs Epochs
3. **Quantitative Evaluation:**
   - Generated image samples
   - Comparison of diversity between models
   - Frechet Inception Distance (FID) → Optional
   - Inception Score (IS) → Optional
4. **App Deployment:**
   - Build a Gradio or Streamlit app for the entire system that demonstrates both models, enables real-world testing, and highlights the differences between them.

Question 2: Doodle-to-Real Image Translation and Colorization using Pix2Pix

Objective:

The objective of this question is to design and implement a paired image-to-image translation system using a conditional GAN, specifically Pix2Pix, to convert:

- Sketch / Edge images → Realistic images
- Grayscale images → Colored images

The model must learn a mapping between paired input-output images and generate visually realistic outputs while preserving structural information.

1. Environment Setup

- **Platform:** https://www.kaggle.com/
- **Accelerator:** GPU T4 x2 (Dual GPU)
- **Dataset:**
  - CUHK Face Sketch Dataset: https://www.kaggle.com/datasets/arbazkhan971/cuhk-face-sketch-database-cufs
  - Anime Sketch Colorization Dataset: https://www.kaggle.com/datasets/ktaebum/anime-sketch-colorization-pair

2. Model Architecture

- **Model: Pix2Pix (Conditional GAN)**
  - **Generator (U-Net Architecture)**

    Configuration:
    - Encoder-Decoder structure
    - Skip connections between encoder and decoder layers
    - Input: Sketch / Edge / Grayscale image
    - Output: Realistic / Colored image

Generator Requirements:

- Preserve spatial structure of input image
- Generate high-quality realistic outputs
- Maintain fine details using skip connections

➢ **Discriminator (PatchGAN)**

Configuration:

- Patch-based classification (16 × 16 patches)
- Classifies whether each patch is real or fake
- Output: Matrix of probabilities

Generator Requirements:

- Focus on local image realism
- Improve texture quality
- Distinguish real vs generated images

3. Implementation Tasks

Part 1: Data Preparation

1. Load paired datasets (input, target images)
2. Resize all images to 256 × 256
3. Normalize images to range [-1, 1]
4. Create PyTorch DataLoader

Part 2: Forward Pass

1. Input sketch image into Generator
2. Generate output image (fake)
3. Pass real pair (input, real image) to Discriminator
4. Pass fake pair (input, generated image) to Discriminator
5. Compute adversarial loss

Part 3: Training Setup

Loss Function

A. Adversarial Loss (GAN Loss)
B. L1 Loss (Reconstruction Loss)
C. Total Loss

Optimizer

- Adam
- Learning Rate: 0.0002
- Betas: (0.5, 0.999)

Training Strategy:

- Train Generator and Discriminator alternately
- Use paired data for supervised learning
- Monitor both adversarial and L1 loss

Required Training Techniques (to fit Kaggle T4×2)

- Mixed Precision (torch.cuda.amp)
- Batch Size: 16–32 (based on GPU memory)
- Reduce image size if needed (e.g., 128 × 128)
- Save checkpoints every 5–10 epochs
- Use dataset subsets for faster training

4. Visualization Module

Create a utility that shows:

- Input Sketch / Edge Image
- Generated Output Image
- Ground Truth Image

Display at least 5-10 generated samples per model.

Deliverables

1. **Jupyter Notebook:**
   - Complete Pix2Pix implementation in PyTorch.
2. **Training Logs:**
   - Plot Generator Loss vs Epochs.
   - Plot Discriminator Loss vs Epochs
3. **Quantitative Evaluation:**
   - Structural Similarity Index (SSIM)
   - Peak Signal-to-Noise Ratio (PSNR)
   - Visual comparison of Input vs Output vs Ground Truth → Optional
4. **App Deployment:**
   - Build a Gradio or Streamlit app for:
     - Accepts sketch / grayscale input
     - Generates realistic or colored image
     - Displays results in real-time

Question 3: Domain Adaptation and Unpaired Image-to-Image Translation using CycleGAN

Objective:

The objective of this question is to design and implement an unpaired image-to-image translation system using CycleGAN to learn mappings between two different domains without paired data. The model must:

- Translate Sketch → Photo
- Translate Photo → Sketch
- Preserve structural consistency using cycle constraints

The model must learn a mapping between paired input-output images and generate visually realistic outputs while preserving structural information.

1. Environment Setup

- **Platform:** https://www.kaggle.com/
- **Accelerator:** GPU T4 x2 (Dual GPU)
- **Dataset:**
  - TU-Berlin Sketch Dataset: https://huggingface.co/datasets/sdiaeyu6n/tu-berlin
  - Sketchy Dataset: https://www.kaggle.com/datasets/sharanyasundar/sketchy-dataset
    - Explore its Readme.md to get the dataset information.
  - Google QuickDraw Dataset: https://www.kaggle.com/c/quickdraw-doodle-recognition/data

2. Model Architecture

- **Model: CycleGAN**
  - **Generator**
    1. **G_AB:** Sketch → Photo
    2. **G_BA:** Photo → Sketch

Configuration:

- Architecture: ResNet-based Generator
- Number of ResNet Blocks: 6 (optimized for Kaggle)
- Image Size: 128 × 128

Generator Requirements:

- Learn mapping between domains
- Preserve structure during translation
- Enable cyclic reconstruction

- **Discriminator (PatchGAN)**
  1. **D_A:** Classifies Sketch domain
  2. **D_B:** Classifies Photo domain

Configuration:

- PatchGAN discriminator
- Classifies image patches instead of full image

Generator Requirements:

- Distinguish real vs fake images
- Improve realism of generated outputs

3. Implementation Tasks

Part 1: Data Preparation

1. Load unpaired datasets (Domain A: Sketch, Domain B: Photo)
2. Resize all images to 128 × 128
3. Normalize images to range [-1, 1]
4. Create separate DataLoaders for each domain

Part 2: Forward Pass

1. Translate Sketch → Photo using G_AB
2. Translate back Photo → Sketch using G_BA
3. Translate Photo → Sketch and back
4. Compute cycle consistency

Part 3: Training Setup

Loss Function

D. Adversarial Loss

A. Cycle Consistency Loss

B. Identity Loss

Optimizer

- Adam
- Learning Rate: 0.0002
- Betas: (0.5, 0.999)

Training Strategy:

- Train both generators and discriminators alternately
- Maintain balance between domains
- Monitor cycle loss closely

Required Training Techniques (to fit Kaggle T4×2)

- Mixed Precision (torch.cuda.amp)
- Batch Size: 4–8 (due to higher memory usage)
- Reduce image size if needed
- Use only subset of dataset

- Reduce ResNet blocks (6 instead of 9)
- Save checkpoints frequently

4. Visualization Module

Create a utility that shows:

- Input Sketch / Edge Image
- Generated Output Image
- Ground Truth Image

Display at least 5 qualitative examples.

Deliverables

1. **Jupyter Notebook:**
   - Complete CycleGAN implementation in PyTorch.
2. **Training Logs:**
   - Plot Generator Loss vs Epochs.
   - Plot Discriminator Loss vs Epochs
   - Cycle Consistency Loss vs Epochs
3. **Quantitative Evaluation:**
   - Structural Similarity Index (SSIM)
   - Peak Signal-to-Noise Ratio (PSNR)
   - Visual comparison of Input image vs Translated Output vs Reconstructed Image → Optional
4. **App Deployment:**
   - Build a Gradio or Streamlit app for:
     - Accepts image input (sketch or photo)
     - Performs domain translation
     - Displays output in real-time